

Flashcard Generator

Vorlesung: Web Services

Professor: Prof. Dr. Alexander Auch



Table of Contents

Einleitung.....	3
Organisation	3
Modelle.....	3
Erstes Modell.....	3
BPMN.....	4
Implementierung.....	4
Frontend.....	4
Verwaltungs Service.....	5
Einleitung:.....	5
Architekturübersicht.....	5
Ablauf der Verarbeitung	5
POST-Request	7
PDF Extraction Service	8
Einleitung.....	8
Architekturübersicht.....	8
Ablauf der Verarbeitung	8
POST-Request	10
Image Description Service.....	11
Einleitung.....	11
Architekturübersicht.....	11
Ablauf der Verarbeitung	11
POST-Request	12
Generate Cards Service	13
Einleitung.....	13
Architekturübersicht.....	13
Ablauf der Verarbeitung	13
POST-Request	15

Einleitung

Der Web-Service Flashcard Generator ist als nützliche Anwendung für Schüler und Studenten gedacht, welcher PDF-Dateien in kleine Karteikarten umwandelt, welche durch Fragen und Antworten die Vorbereitung auf Klausuren erleichtern. Der Service verwendet KI-Services, um richtige Fragen und Antworten für die Flashcards zu generieren.

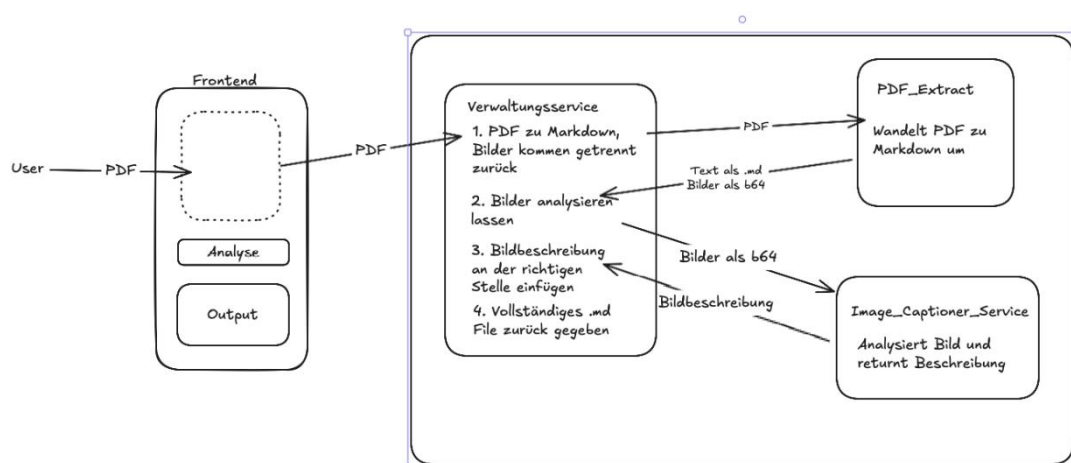
Organisation

Name	Matrikelnummer
Louis Hauger	9796557
Katharina Heller	9207280
Johannes Kommer	2576582
Massimo Keil	3181933

Bearbeitungsschritt	Verantwortlich
Frontend	Katharina Heller
Flask Services	Louis Hauger
BPNM	Johannes Kommer
Swagger UI	Johannes Kommer
Dokumentation	Massimo Keil
Präsentation	Katharina Heller
Deployment	Massimo Keil

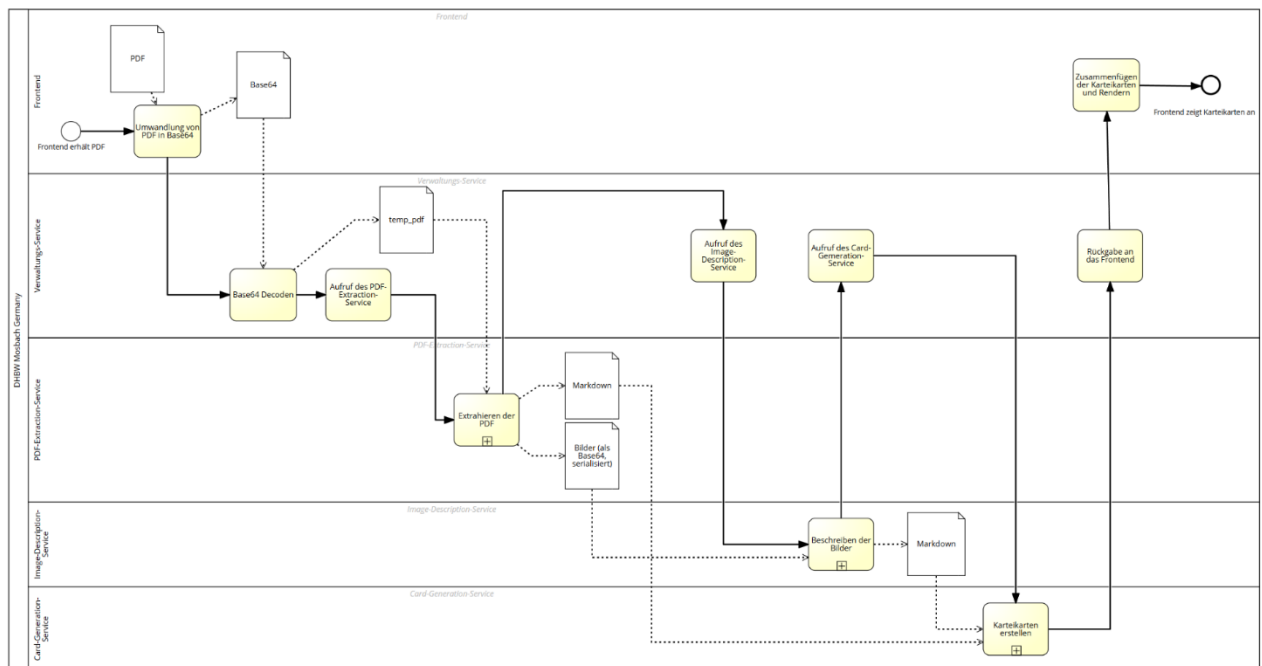
Modelle

Erstes Modell



Unser erstes Modell, welches die grobe Funktionsweise unseres Flashcard Generators beschreibt. Im Verlauf der Entwicklung wurden einige Änderungen, wie das Einführen eines Generate Card Services und die Umwandlung in Base64 vom Frontend.

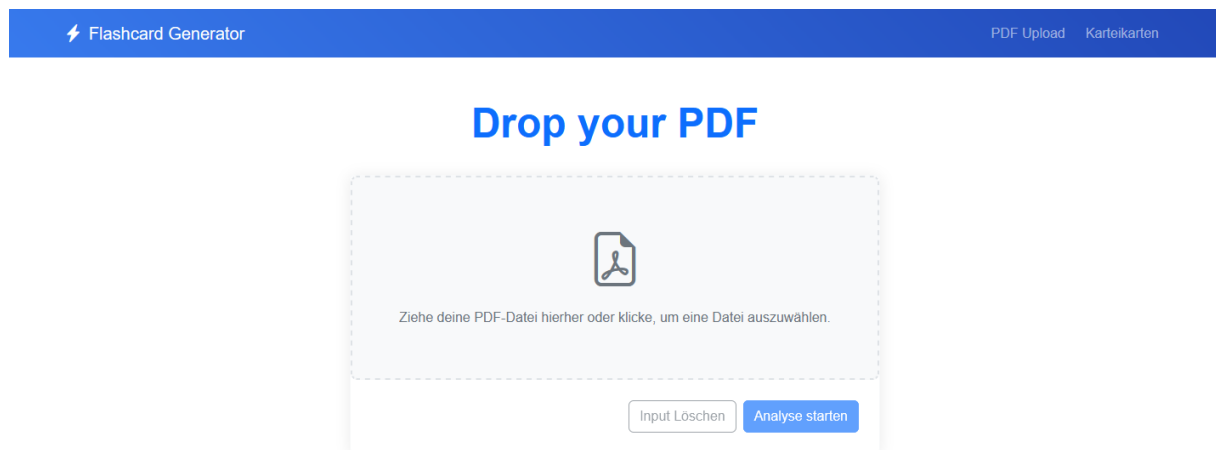
BPMN



Unser BPMN zeigt genau auf wie der Ablauf unseres Finalen Service aussieht.

Implementierung

Frontend



Unser Frontend nutzt ein simples Design zum verwenden unseres Services. Vom Hauptbildschirm aus kann man:

- Ein PDF per Auswahl oder Drag-And-Drop einfügen
- Diese wieder entfernen
- Die Analyse des PDFs starten

Verwaltungs Service

Einleitung:

Der bereitgestellte Flask-Service ist eine API, die PDF-Dateien analysiert und Flashcards erstellt. Der Service nimmt eine PDF-Datei in Base64-kodierter Form entgegen, extrahiert den Inhalt und verarbeitet Bilder, um sie zu beschreiben. Dafür nutzt er den **PDF Extraction Service**. Die extrahierten Bilder werden dann mit dem **Image Description Service** beschrieben und die Beschreibung wird zu der ursprünglichen Position des Bildes eingefügt. Die verarbeitete PDF geht zuletzt in den **Generate Cards Service** um die Flashcards zu generieren.

Architekturübersicht

1. Empfang der PDF-Datei (POST Request)
2. Dekodierung der Base64-kodierten PDF
3. Speichern der PDF als temporäre Datei
4. Aufruf des PDF-Extraktionsservices
5. Verarbeitung von Bildern im PDF
6. Ersetzung der Bildreferenzen mit Beschreibungen
7. Aufruf des Flashcards-Services zur Erstellung von Flashcards
8. Rückgabe der generierten Flashcards als Antwort

Ablauf der Verarbeitung

1. Empfang der PDF-Datei (POST Request)

```
14 @app.route(rule: '/analyse-pdf', methods=['POST'])  CookieClickr +1
15 def analyse_pdf():
16     data = request.get_json()
17     if not data or 'pdf' not in data:
18         return jsonify({"error": "Missing 'pdf' field in JSON payload"}), 400
19
20     # PDF dekodieren
21     try:
22         pdf_bytes = base64.b64decode(data['pdf'])
23     except Exception as e:
24         return jsonify({"error": "Invalid Base64 for PDF", "details": str(e)}), 400
25
26     # Temporäre Datei anlegen
27     with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as temp_pdf:
28         temp_pdf.write(pdf_bytes)
29         temp_pdf_path = temp_pdf.name
```

- Die PDF-Datei wird empfangen
- Umwandlung in Base64

- Speichern als temporäre Datei

2. Aufruf des PDF Extraction Service

```

31     # PDF-Extraction-Service aufrufen
32     try:
33         with open(temp_pdf_path, 'rb') as f:
34             files = {'file': f}
35             pdf_extract_response = requests.post(
36                 "http://pdf-extraction-service:5003/pdf-extract",
37                 files=files
38             )
39     except Exception as e:
40         if os.path.exists(temp_pdf_path):
41             os.remove(temp_pdf_path)
42         return jsonify({"error": "Error calling PDF-Extraction-Service", "details": str(e)}), 500
43
44     # Temporäre Datei löschen
45     if os.path.exists(temp_pdf_path):
46         os.remove(temp_pdf_path)
47
48     if pdf_extract_response.status_code != 200:
49         return jsonify({
50             "error": "PDF Extraction Service error",
51             "details": pdf_extract_response.text
52         }), 500
53
54     extract_data = pdf_extract_response.json()
55     markdown = extract_data.get("markdown", "")
56     images = extract_data.get("images", {})

```

- Temporäre Datei an den PDF Extraction Service
- Erhalten von Markdown und Bildern

3. Verarbeitung von Bildern mit Image Description Service

```

58     # Für jedes Bild den Image-Desc-Service aufrufen und Markdown ersetzen
59     for filename, img_info in images.items():
60         base64_str = img_info.get("data", "")
61
62         try:
63             img_desc_response = requests.post(
64                 "http://image-description-service:5002/describe_image",
65                 json={"filename": filename, "data": base64_str}
66             )
67         except Exception as e:
68             desc = "Fehler bei Bildbeschreibung"
69         else:
70             if img_desc_response.status_code == 200:
71                 desc_data = img_desc_response.json()
72                 desc = desc_data.get("image_desc", {}).get(filename, {}).get("data", "Keine Beschreibung verfügbar")
73             else:
74                 desc = "Keine Beschreibung verfügbar"
75

```

- Bilder werden nach und nach beschrieben

4. Erstellen der Flashcards mit Generate Cards Service

```

79     # Aufruf des generate_cards_service
80     try:
81         generate_cards_response = requests.post(
82             " http://generate-cards-service:5001/generate_cards",
83             json={"markdown_text": markdown}
84         )
85     except Exception as e:
86         return jsonify({"error": "Error calling Generate Cards Service", "details": str(e)}), 500
87
88     if generate_cards_response.status_code != 200:
89         return jsonify({
90             "error": "Generate Cards Service error",
91             "details": generate_cards_response.text
92         }), 500
93
94     flashcards = generate_cards_response.json()
95
96     return jsonify({
97         "flashcards": flashcards
98     }), 200

```

- Flashcard werden Generiert und zurück gegeben

POST-Request

Eingabe

```

{
  "file": "<Base64-codierte PDF-Daten>"
}

```

Antwort

```

{
  {
    "title": "Thema 1",
    "front": "Was ist ein wichtiger Fakt?",
    "back": "Hier steht ein wichtiger Fakt."
  },
  {
    "title": "Thema 2",
    "front": "Welche Information gibt es?",
    "back": "Eine weitere Information."
  }
}

```

PDF Extraction Service

Einleitung

Der PDF Extraction Service verarbeitet hochgeladene PDF-Dateien und extrahiert deren Inhalt als Markdown sowie eingebettete Bilder im Base64-Format.

Architekturübersicht

1. Eingang einer PDF-Datei per API-Request
2. Verarbeitung mit marker und Extraktion von Text und Bildern
3. Umwandlung von Bildern in Base64
4. Rückgabe der extrahierten Daten als JSON

Ablauf der Verarbeitung

1. PDF-Converter

```
# Konfiguration für den PdfConverter
config = {
    "output_format": "markdown",
    "languages": "de",
    "use_fast": True
}
config_parser = ConfigParser(config)

converter = PdfConverter(
    config=config_parser.generate_config_dict(),
    artifact_dict=create_model_dict(),
    processor_list=config_parser.get_processors(),
    renderer=config_parser.get_renderer()
)
```

- PDF Converter wird konfiguriert

2. Umwandlung der Bilder


```
def serialize_image(image): 1 usage  CookieClickr
    """
    Konvertiert ein PIL.Image-Objekt in einen Base64-String und packt ihn in ein Dictionary
    mit dem Key 'data'.
    """
    if image.mode not in ("RGB", "RGBA"):
        image = image.convert("RGB")
    with BytesIO() as buffer:
        image.save(buffer, format="PNG")
        image_bytes = buffer.getvalue()
    base64_str = base64.b64encode(image_bytes).decode('utf-8')
    return {"data": base64_str}
```

- Bilder werden in Base64 umgewandelt

3. API-Route für PDF Extract

```
@app.route(rule='/pdf-extract', methods=['POST'])  CookieClickr
def extract_pdf():
    """
    Verarbeitet eine hochgeladene PDF-Datei und gibt Markdown und Bilder (Base64) zurück.
    {
        "markdown": "Inhalt ...",
        "images": {
            "page_0_Figure_1.jpeg": {"data": "<base64>"},
            "page_1_Figure_2.jpeg": {"data": "<base64>"}
        }
    }
    """
    if 'file' not in request.files:
        return jsonify({"error": "Keine Datei im Request gefunden"}), 400

    pdf_file = request.files['file']
    if not pdf_file.filename.lower().endswith('.pdf'):
        return jsonify({"error": "Bitte eine PDF-Datei hochladen."}), 400

    temp_path = os.path.join('/tmp', pdf_file.filename)
    try:
        pdf_file.save(temp_path)
        rendered = converter(temp_path)
        if not rendered:
            return jsonify({"error": "Unbekanntes Ausgabeformat"}), 500

        # Bilder serialisieren
        img_dict = rendered.images
        serialized_images = {}
        if img_dict:
```

```

86         for key, img in img_dict.items():
87             if isinstance(img, Image.Image):
88                 serialized_images[key] = serialize_image(img)
89             else:
90                 serialized_images[key] = {"data": ""}
91         else:
92             serialized_images = {}
93
94         result = {
95             "markdown": rendered.markdown,
96             "metadata": rendered.metadata,
97             "images": serialized_images
98         }
99         return jsonify(result), 200
100
101     except Exception as e:
102         return jsonify({"error": str(e)}), 500
103
104     finally:
105         if os.path.exists(temp_path):
106             os.remove(temp_path)
107
108 if __name__ == '__main__':
109     app.run(host="0.0.0.0", port=5003, debug=True)
110

```

- Diese Route empfängt eine PDF-Datei, verarbeitet sie und gibt den extrahierten Text sowie die Bilder zurück.

POST-Request

Eingabe

```

{
    "file": "<PDF-Datei>"
}

```

Antwort

```

{
    "markdown": "Inhalt des PDFs als Markdown-Text",
    "metadata": {"Titel": "Dokumenttitel", "Autor": "Autornamen"},
    "images": {
        "page_0_Figure_1.jpeg": {"data": "<Base64-kodiertes Bild>"},
        "page_1_Figure_2.jpeg": {"data": "<Base64-kodiertes Bild>"}
    }
}

```

Image Description Service

Einleitung

Der "Image Description Service" ermöglicht die automatische Beschreibung von Bildern durch die API von Google Gemini. Der Service empfängt ein Base64-kodiertes Bild und gibt eine detaillierte Beschreibung des Inhalts zurück.

Architekturübersicht

1. Empfangen eines Base64-kodierten Bildes per API-Request
2. Speichern des Bildes temporär als Datei
3. Analyse des Bildes mit Google Gemini API
4. Rückgabe der generierten Bildbeschreibung als JSON

Ablauf der Verarbeitung

1. Speichern des Bildes

```
17 def save_temp_image(img_b64): 1 usage  📄 CookieClickr
18     image_data = base64.b64decode(img_b64)
19     image = Image.open(BytesIO(image_data))
20
21     with tempfile.NamedTemporaryFile(suffix=".jpg", delete=False) as temp_file:
22         image.save(temp_file, format="JPEG")
23     return temp_file.name
```

2. API-Route für die Bildbeschreibung

```

26 @app.route(rule: "/describe_image", methods=["POST"])  👤 CookieClickr
27 def describe_image():
28     try:
29         data = request.json
30         if "filename" not in data or "data" not in data:
31             return jsonify({"error": "Invalid input format"}), 400
32
33         image_path = save_temp_image(data["data"])
34
35         response = client.models.generate_content(
36             model='gemini-2.0-flash',
37             contents=[
38                 'Describe this image in about 100 words.',
39                 Image.open(image_path)
40             ]
41         )
42
43         os.remove(image_path) # Temporäre Datei löschen
44
45         return jsonify({
46             "image_desc": {
47                 data["filename"]: {"data": response.text}
48             }
49         })
50
51     except Exception as e:
52         return jsonify({"error": str(e)}), 500

```

- Diese Route nimmt ein Bild entgegen und gibt eine Beschreibung zurück.

POST-Request

Eingabe

```

{
  "filename": "bild1.jpg",
  "data": "iVBORw0KGgoAAAANSUUEUgAA... (Base64-kodiertes Bild)"
}

```

Antwort

```

{
  "image_desc": {

```

```
"bild1.jpg": {  
  "data": "This is a picture of a beautiful sunset over the mountains, with vibrant orange  
and pink hues filling the sky. The sun is partially hidden behind the peaks, casting long shadows  
on the landscape below."  
}
```

Generate Cards Service

Einleitung

Der "Generate Cards Service" ist ein Flask-gestützter Webservice, der Markdown-Text als Eingabe entgegennimmt und daraus automatisch Karteikarten generiert. Dies erfolgt mithilfe der OpenAI GPT-API, die strukturierte JSON-Antworten liefert.

Architekturübersicht

1. Eingang eines Markdown-Texts per API-Request
2. Aufteilung des Textes in verarbeitbare Abschnitte (Chunks)
3. Generierung von Karteikarten durch OpenAI API
4. Zusammenführung der Ergebnisse und Rückgabe als JSON

Ablauf der Verarbeitung

1. Aufteilung des Textes in kleinere Chunks

```
32 def split_markdown(markdown_text: str, max_chunk_size: int = MAX_CHUNK_SIZE) -> list:  
33     sections = []  
34     current_section = []  
35     for line in markdown_text.splitlines():  
36         if line.startswith('# ') and current_section:  
37             sections.append("\n".join(current_section))  
38             current_section = [line]  
39         else:  
40             current_section.append(line)  
41     if current_section:  
42         sections.append("\n".join(current_section))
```

- Der Markdown-Text wird anhand von Überschriften (#) in Abschnitte unterteilt.
- Jeder Abschnitt wird weiter in kleinere Blöcke zerlegt, falls nötig.

2. Erstellung der Prompts für die KI

```

63 def create_system_prompt() -> str: 1 usage  🧑 CookieClickr
64     return ""
65     Du bist ein Assistent, der aus einem gegebenen Markdown-Text Karteikarten generiert. Beachte folgende Anweisungen:
66
67     1) Erstelle Karteikarten im Format 'Frage: Antwort' mit klaren, abgeschlossenen Einheiten.
68     2) Inhaltsverzeichnisse, Metadaten zu Kursen oder Quellenverzeichnisse sind nicht relevant.
69     3) Jede Karteikarte hat einen Titel, eine Vorderseite und eine Rückseite.
70     4) Die Antwort erfolgt ausschließlich als JSON, IN GÜLTIGEM JSON FORMAT, ohne zusätzlichen Text.
71     Format:
72     {
73         "title": "Titel der Karte",
74         "front": "Frage/Problemstellung",
75         "back": "Erklärung/Lösung"
76     }
77     ""

```

- Hier wird der Systemprompt definiert, um die OpenAI API präzise anzuweisen.

```

80 def create_user_prompt(markdown_chunk: str) -> str: 1 usage  🧑 CookieClickr
81     return f"""
82     ### Hier ist der Markdown-Text:
83     {markdown_chunk}
84     """

```

- Der User-Prompt enthält den spezifischen Markdown-Abschnitt, der analysiert werden soll.

3. Verarbeitung eines Text-Chunks mit OpenAI

```

87 async def process_chunk(markdown_chunk: str): 1 usage  🧑 CookieClickr
88     system_prompt = create_system_prompt()
89     user_prompt = create_user_prompt(markdown_chunk)
90     try:
91         completion = await client.beta.chat.completions.parse(
92             model="gpt-4o-2024-08-06",
93             messages=[
94                 {"role": "system", "content": system_prompt},
95                 {"role": "user", "content": user_prompt}
96             ],
97             response_format=Flashcard,
98         )
99         flashcard = completion.choices[0].message
100         if flashcard.refusal:
101             return None
102         else:
103             return flashcard.parsed
104     except Exception as e:
105         return None

```

- Asynchrone API-Anfrage an OpenAI zur Erzeugung von Karteikarten.

- Fehlerbehandlung sorgt dafür, dass ein fehlgeschlagener API-Call nicht den gesamten Prozess stoppt.

4. Verarbeiten aller Markdown-Chunks

```
108     async def generate_cards(markdown_text: str): 1 usage  👤 CookieClickr
109         chunks = split_markdown(markdown_text)
110         tasks = [process_chunk(chunk) for chunk in chunks]
111         results = await asyncio.gather(*tasks)
112         return [result for result in results if result is not None]
```

- Alle Text-Chunks werden parallel an die OpenAI API gesendet.
- Die Ergebnisse werden gesammelt und als Liste zurückgegeben.

5. API-Route für den Webservice

```
115 @app.route(rule: "/generate_cards", methods=["POST"])  👤 CookieClickr
116 def generate_flashcards():
117     data = request.json
118     if "markdown_text" not in data:
119         return jsonify({"error": "Fehlende 'markdown_text' im Request"}), 400
120
121     markdown_text = data["markdown_text"]
122     loop = asyncio.new_event_loop()
123     asyncio.set_event_loop(loop)
124     flashcards = loop.run_until_complete(generate_cards(markdown_text))
125
126     return jsonify([card.model_dump() for card in flashcards])
```

- Diese Route nimmt einen Markdown-Text per POST-Request entgegen.
- Der Text wird verarbeitet, und die generierten Karteikarten werden als JSON zurückgegeben.

POST-Request

Eingabe

```
{
  "markdown_text": "# Thema 1\nHier steht ein wichtiger Fakt.\n\n# Thema 2\nEine weitere Information."
}
```

Antwort

```
[
  {
    "title": "Thema 1",
    "front": "Was ist ein wichtiger Fakt?",
    "back": "Hier steht ein wichtiger Fakt."
  },
  {
    "title": "Thema 2",
    "front": "Welche Information gibt es?",
    "back": "Eine weitere Information."
  }
]
```